Lojik Devreler ve Tasarım

Bahar 2023

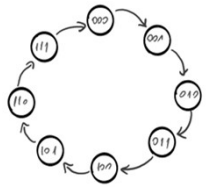
Dr. Öğr. Ü. İhsan Çiçek

11. DTD: Ardışık Lojik Devreler ve Sonlu Durum Makineleri

GEBZE TEKNİK ÜNİVERSİTESİ

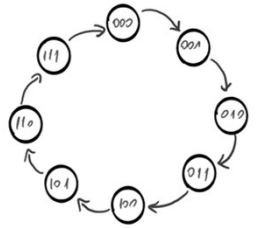
Simple 3-bit Counter

- Let's examine a simple 3-bit binary counter
  - 8 possible values ($2^3 = 8$)
  - We need total of 3 flip-flops to hold the values
  - Can represent any value between 000 - 111
- E.g. For counting up to 1000 we need at least 10 ffs ($2^{10} > 1000$
- We can represent the transition of outputs of ffs with bubbles.



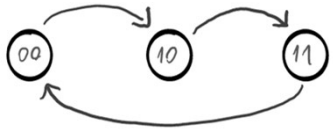
State Transition Diagram

- **state transition diagram:** the diagram that shows the ff output transitions
- Each **bubble** is a **state** that shows the ff outputs
- Each **arrow** is a **clock edge**
- Each **number** inside a state is the **ff outputs**
- Example shows a 3-bit binary counter states



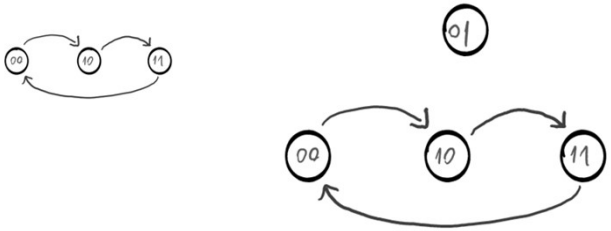
State Properties 1

- It does not have to count in order
- It does not have to include all possible outputs as states



State Properties 2

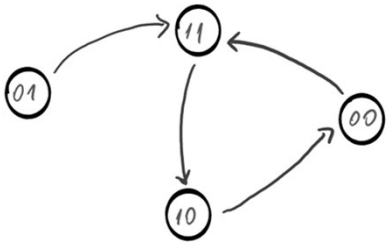
- It does not have to count in order
- It does not have to include all possible outputs as states
- Following are equivalent



GEBZE TEKNİK ÜNİVERSİTESİ 5

State Properties 3

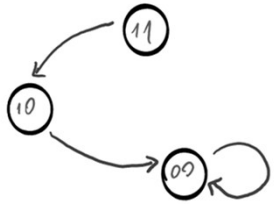
- It does not have to return to each state.



GEBZE TEKNİK ÜNİVERSİTESİ 6

State Properties 4

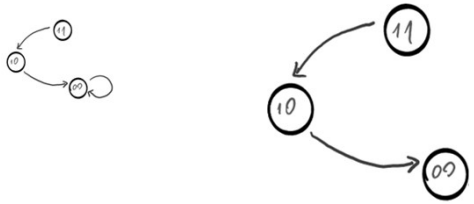
- It does not have to return to each state.
- It can stay in a given state



GEBZE TEKNİK ÜNİVERSİTESİ 7

State Properties 5

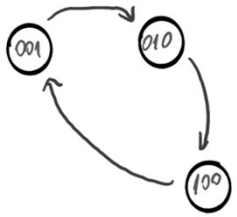
- It does not have to return to each state.
- It can stay in a given state
- If that loopback arrow is not shown, the assumption is it stays in that state



GEBZE TEKNİK ÜNİVERSİTESİ 8

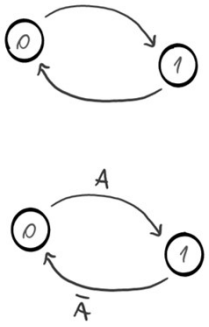
State Properties 6

- It does not need to have minimum number of ffs possible



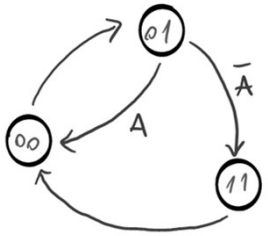
State Properties 7

- Transitions can depend on input(s)
- When FF is in 0 state
  - if A input is 1 transition to 1 state
- When FF is in 1 state
  - if A input is 0 transition to 0 state



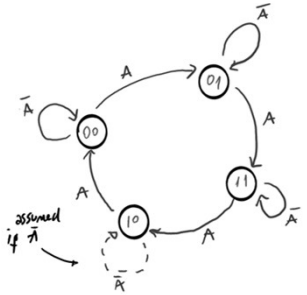
State Properties 8

- All transitions does not need to include (all) inputs
- If there is only arrow, it means when the clock edge hits, state transition will happen



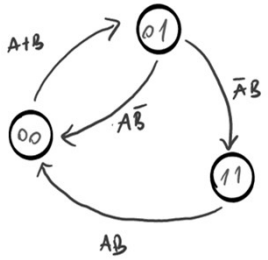
State Properties 9

- Loops also does not need to be drawn
  - If A is 1, transition to next state
  - If A is 0, stay in the same state



State Properties 10

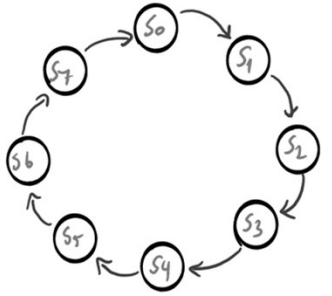
- There can be multiple inputs

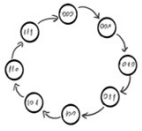


GEBZE TEKNİK ÜNİVERSİTESİ 13

State Properties 11

- States can be encoded with unique names

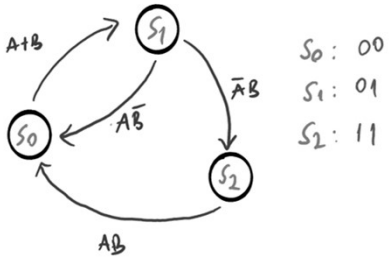




GEBZE TEKNİK ÜNİVERSİTESİ 15

State Properties 11

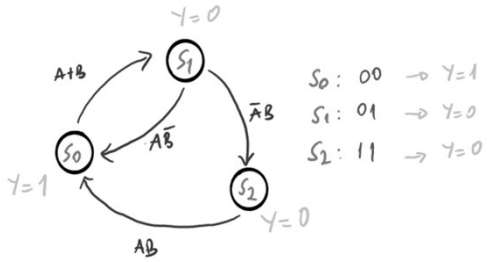
- States can be encoded with unique names



GEBZE TEKNİK ÜNİVERSİTESİ 14

State Properties 12

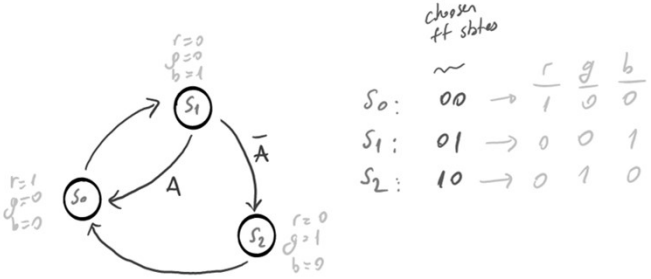
- There can be an output (aside from ff outputs, and based on ff outputs) in each state



GEBZE TEKNİK ÜNİVERSİTESİ 16

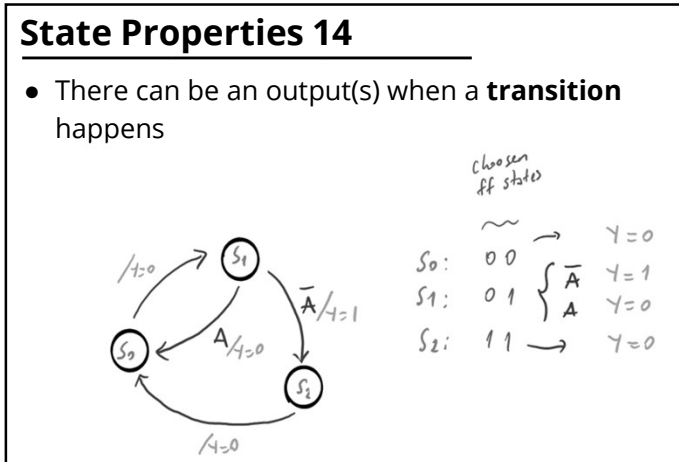
State Properties 13

- There can be multiple outputs



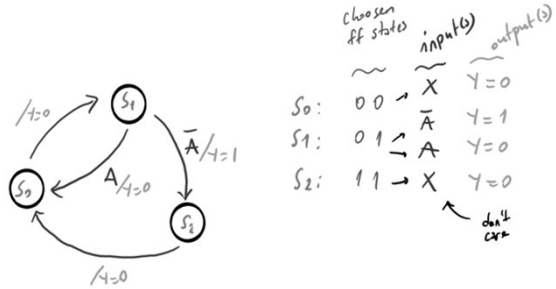
State Properties 14

- There can be an output(s) when a **transition** happens



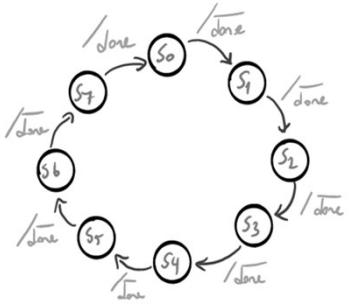
State Properties 14

- There can be an output(s) when a **transition** happens



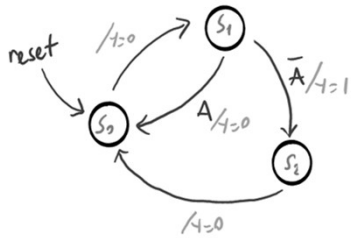
State Properties 14

- There can be an output(s) when a **transition** happens



State Properties 15

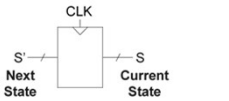
- **reset** state can be denoted with a **reset** input coming into that state

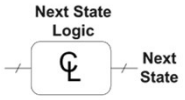


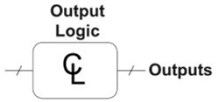
Finite State Machine (FSM)

Consists of:

- **State register**
  - Stores current state
  - Loads next state at clock edge
- **Combinational logic**
  - Computes the next state
  - Computes the outputs

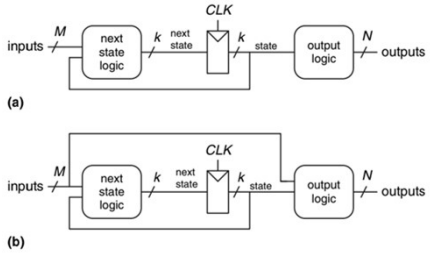






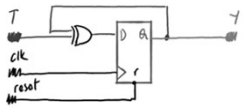
Finite State Machines (FSM)

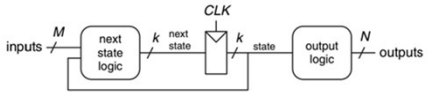
- **Next state** determined by current state and inputs
- Two types of finite state machines differ in **output logic**:
  - **Moore FSM:** outputs depend only on current state
  - **Mealy FSM:** outputs depend on current state and inputs



FSM Example - T Flip Flop

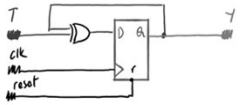
- Recall that T Flip Flop consist of a register an an XOR gate
  - Input(s): T, clk, reset
  - Output(s): Y
- If we try to match this design to one of the FSM types
  - **XOR** gate is the **next-state logic**
  - **register** is the **register** that generates the states
  - **output logic** is a **wire**

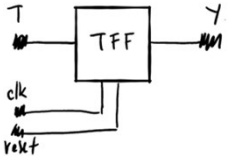




FSM Black Box - T Flip Flop

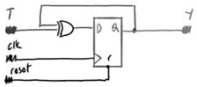
- Let's build it.
  - Input(s): T, clk, reset
  - Output(s): Y





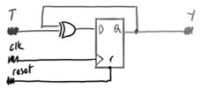
FSM State Transition Diagram - T Flip Flop

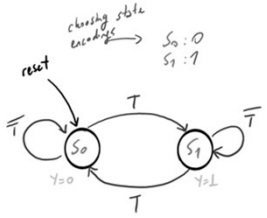
- Let's build state transition diagram
- **Moore FSM:** outputs labeled in each state
- **States:** Circles
- **Transitions:** Arrows



FSM State Transition Diagram - T Flip Flop

- Let's build state transition diagram
- **Moore FSM:** outputs labeled in each state
- **States:** Circles
- **Transitions:** Arrows
- Since there are 2 states, we only need 1 register!

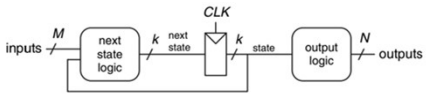


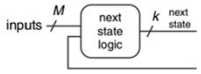


FSM State Transition Table - T Flip Flop

Now that we have the diagram, we will first work on the next state logic.

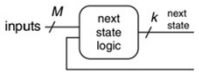
1. Define all the **inputs** to the next state logic
2. Define all the outputs of the next state logic
3. Create a truth table, and find the boolean expressions

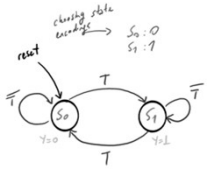




FSM State Transition Table - T Flip Flop

- Now that we have the diagram, let's work on the **next-state logic**.

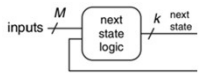


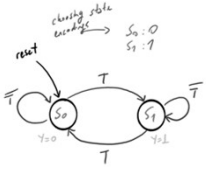


inputs	states	outputs: next-states
T		
0	S_0	S_0
1	S_0	S_1
0	S_1	S_1
1	S_1	S_0

FSM State Transition Table - T Flip Flop

- Now that we have the diagram, let's work on the **next-state logic**.
- Let's plug-in our state encodings

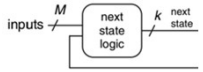




inputs	states	outputs: next-states
T	f_0	f_{0next}
0	S_0 : 0	S_0: 0
1	S_0 : 0	S_1: 1
0	S_1 : 1	S_1: 1
1	S_1 : 1	S_0: 0

FSM State Transition Table - T Flip Flop

- Now that we have the diagram, let's work on the **next-state logic**.
- Let's plug-in our state encodings
- Find boolean expression for next-



inputs	states	outputs: next-states
T	f_0	f_{0next}
0	S_0 : 0	S_0: 0
1	S_0 : 0	S_1: 1
0	S_1 : 1	S_1: 1
1	S_1 : 1	S_0: 0

f_{0next} =

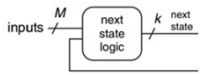
f_0 \ T	0	1
0	0	1
1	1	0

$f_{0next} = T\bar{f}_0 + \bar{T}f_0$

$f_{0next} = T \oplus f_0$

FSM State Transition Table - T Flip Flop

- Now that we have the diagram, let's work on the **next-state logic**.
- Let's plug-in our state encodings
- Find boolean expression for next-



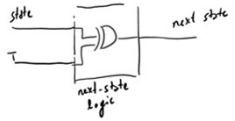
inputs	states	outputs: next-states
T	f_0	f_{0next}
0	S_0 : 0	S_0: 0
1	S_0 : 0	S_1: 1
0	S_1 : 1	S_1: 1
1	S_1 : 1	S_0: 0

f_{0next} =

f_0 \ T	0	1
0	0	1
1	1	0

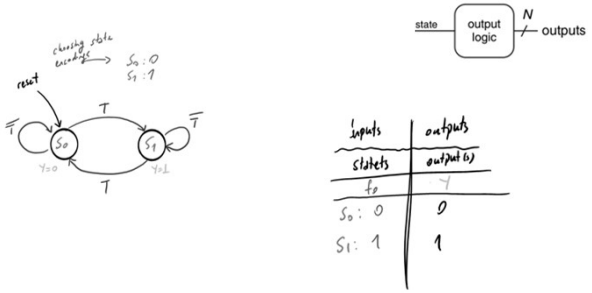
$f_{0next} = T\bar{f}_0 + \bar{T}f_0$

$f_{0next} = T \oplus f_0$



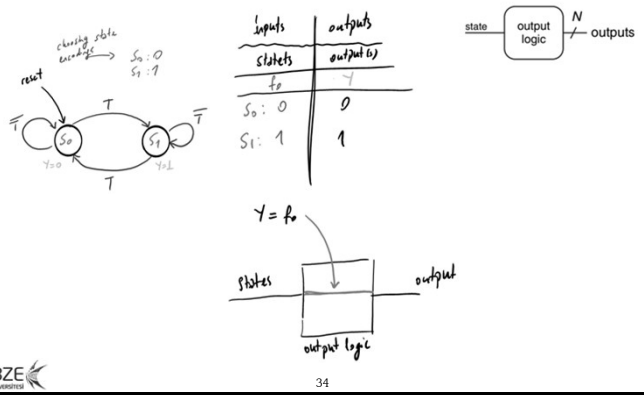
FSM Output Table - T Flip Flop

- Let's work on the last piece of the puzzle, the **output logic**.



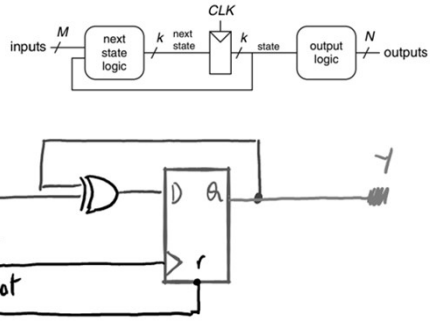
FSM Output Table - T Flip Flop

- Let's work on the last piece of the puzzle, the **output logic**.



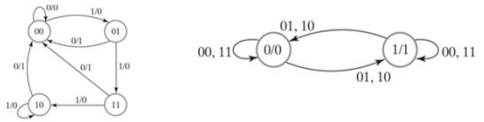
FSM Final - T Flip Flop

- We now have all the pieces. Here is the final circuit



SEQUENTIAL CIRCUIT ANALYSIS

State Diagrams: The information available in a state table may be represented graphically in the form of a state diagram. A state is represented by a circle, and transitions between states are indicated by directed lines connecting the circles.



Mealy Type Moore Type

The binary number inside each circle identifies the state of the FFs. For Mealy model circuits, the directed lines are labeled with two binary numbers separated by a slash. The input value during the present state precedes the slash, and the value following the slash gives the output value during the present state with the given input applied. Output @ Present state with current Input = I/O over the directed line. In Moore type the the output is included inside the state circle, indicated here with a slash

SEQUENTIAL CIRCUIT ANALYSIS

The state table is easier to derive from a given logic diagram and input equations. The state diagram follows directly from the state table and gives a pictorial view of state transitions and is the form more suitable for human interpretation of the operation of the circuit.

In simulation a set of input patterns must be applied in a sequence which includes timely application of input patterns as well as clock pulses. And there must be some means to place the circuit in a known state (Initialization - Reset). Add a circuit output with a path from each state variable signal to observe. The timing of application of inputs and observation of outputs relative to the active clock edge is important.

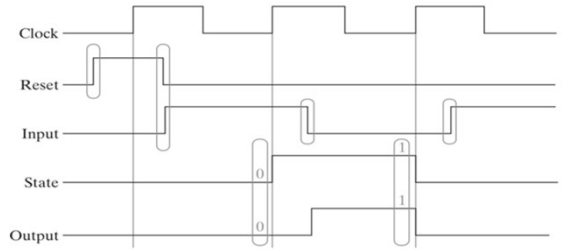
Functional simulation is done for determination or verification of the function of the circuit in which components of the circuit have no delay or a very small delay.

Timing simulation involves realistic delays and verification of the proper operation of the circuit in terms of timing is the objective.

37

SEQUENTIAL CIRCUIT ANALYSIS

The longest delay between the combinational blocks must be smaller than the clock period (critical path). Otherwise circuit operate erroneously. The inputs for a given clock cycle should be changed well before the positive clock edge, preferably early in the clock cycle while the clock is still at a 1 value. The state-variable values and outputs should be at their final values just before the positive clock edge. Although it may be possible to observe the values at other locations, this location provides a foolproof observation time for functional simulation.



38

SEQUENTIAL CIRCUIT DESIGN

A truth table, and a state table is required for a sequential circuit. The minimum number of FFs is determined by the number of states in the circuit; n FFs can represent up to 2^n binary states. The combinational circuit is derived from the state table by finding the FF input equations and output equations. once the type and the number of FFs are determined and binary combinations are assigned to the states, the design process transforms a sequential circuit problem into a combinational circuit problem. In this way, the techniques of combinational circuit design can be applied.

Design Procedure

1. Specification: Write a specification for the circuit, if not already available.

2. Formulation: Obtain either a state diagram or a state table from the statement of the problem.

3. State assignment: If only a state diagram is available from step 1, obtain the state table. Assign binary codes to the states in the table.

39

SEQUENTIAL CIRCUIT DESIGN

Design Procedure

4. Flip-Flop input equation Determination: Select the FF type or types. Derive the FF input equations from the next-state entries in the encoded state table.

5. Output equation Determination: Derive output equations from the output entries in the state table.

6. Optimization: Optimize the FF input equations and output equations.

7. Technology mapping: Draw a logic diagram of the circuit using FFs, ANDs, ORs, and inverters. Transform the logic diagram to a new diagram using the available FF and gate technology.

8. Verification: Verify the correctness of the final design.

40

SEQUENTIAL CIRCUIT DESIGN

Finding State Diagrams

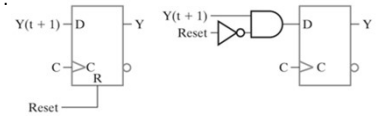
Verbal real world problem has to be interpreted to find a state diagram/table. A state is used to "remember" something about the history of input combinations applied to the circuit either at triggering clock edges or during triggering pulses. State is an abstraction of the sequence of input combinations at the triggering points. There is potential for the set of states to become unnecessarily large or potentially even infinite in size! Instead of adding a new state for every current state and possible applied input combination, it is essential that states be reused as next states to prevent uncontrolled state growth. The mechanism for doing this is a knowledge of the abstraction that each state represents.

41

SEQUENTIAL CIRCUIT DESIGN

Initialization – The Reset

When the power in a digital system is first turned on, the state of the FFs is unknown. It is possible to apply an input sequence with the circuit in an unknown state, but that sequence must be able to bring a portion of the circuit to a known state before meaningful outputs can be expected. The circuits that we design must have a known initial state, and further, a hardware mechanism must be provided to get the circuit from any unknown state into this state. This mechanism is a reset or master reset signal. Regardless of all other inputs applied to the circuit, the reset places the circuit in its initial state. In fact, the initial state is often called the reset state. The reset signal is usually activated automatically when the circuit is powered up. In addition, it may be activated electronically or by pushing a reset button.



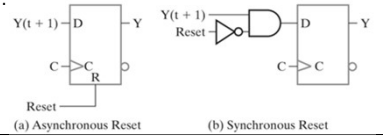
(a) Asynchronous Reset (b) Synchronous Reset

42

SEQUENTIAL CIRCUIT DESIGN

Initialization – The Reset

When the power in a digital system is first turned on, the state of the FFs is unknown. It is possible to apply an input sequence with the circuit in an unknown state, but that sequence must be able to bring a portion of the circuit to a known state before meaningful outputs can be expected. the circuits that we design must have a known initial state, and further, a hardware mechanism must be provided to get the circuit from any unknown state into this state. This mechanism is a reset or master reset signal. Regardless of all other inputs applied to the circuit, the reset places the circuit in its initial state. In fact, the initial state is often called the reset state. The reset signal is usually activated automatically when the circuit is powered up. In addition, it may be activated electronically or by pushing a reset button.

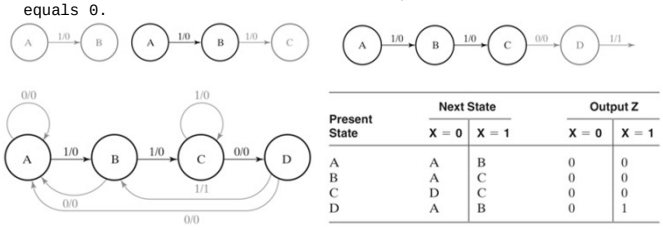


(a) Asynchronous Reset (b) Synchronous Reset

43

SEQUENTIAL CIRCUIT DESIGN

Example: Sequence Detector, a circuit that recognizes the occurrence of a particular sequence of bits, regardless of where it occurs in a longer sequence. This detector has one input X and one output Z. It has Reset applied to the direct reset inputs on its FFs to initialize the state of the circuit to all zeros. The circuit is to recognize the occurrence of the sequence of bits 1101 on X by making Z equal to 1 when the previous three inputs to the circuit were 110 and current input is a 1. Otherwise, Z equals 0.

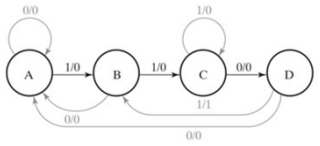


Present State	Next State X = 0	Next State X = 1	Output Z X = 0	Output Z X = 1
A	A	B	0	0
B	A	C	0	0
C	D	C	0	0
D	A	B	0	1

44

SEQUENTIAL CIRCUIT DESIGN

```
module seq_rec_v(
input logic clk, input logic rst, input logic X, output logic Z);
logic [1:0] state, next_state;
//State Encoding
parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;
// state register: implements positive edge- triggered
// state storage with asynchronous reset.
always_ff @(posedge clk or posedge rst) begin
if (rst) state <= A; //Initial state on reset
else state <= next_state;
end
//Next state as function of X and state
always_comb @(X or state) begin
case (state)   // X=1 : X=0
A: next_state = X ? B : A;
B: next_state = X ? C : A;
C: next_state = X ? C : D;
D: next_state = X ? B : A;
default: next_state = A;
endcase
end
//Continued on next slide…
```



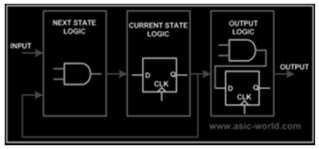
45

SEQUENTIAL CIRCUIT DESIGN

Creating FSMs in SystemVerilog
1. We need the **state register** to hold the current state -> always @(posedge clk) block.

2. We also need **next state function** which defines where we go from each state given the inputs. State by state case analysis: Next state is determined by current state and inputs

3. We need the **output function** to perform necessary acions. State by state analysis. In Moore machines output is determined by current state only, in Mealy machines output is determined by current state and inputs



47

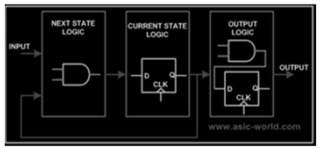
SEQUENTIAL CIRCUIT DESIGN

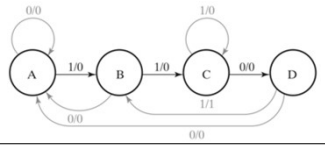
```
//…from previous slide.
// output function: implements output as function of X and state

always_comb @(X or state)
begin //defaults


case (state)
A: Z = 1'b0;
B: Z = 1'b0;
C: Z = 1'b0;
D: Z = X ? 1'b1 : 1'b0;
default: Z = 0;
endcase

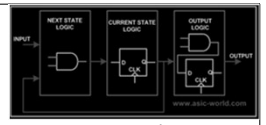

end
endmodule
```





46

SEQUENTIAL CIRCUIT DESIGN



Creating FSMs in SystemVerilog
1. State Register: state variable holds the current state (output of the state register) next_state variable is the evaluated next state value (input of the state register) and assigned to (current) state variable on the posedge of clk.

We depend on next_state combinational function to give us the next state.

Declare symbols for states with state assignment (State Encoding)

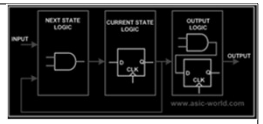
```
parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

logic [1:0] state; //current state
logic [1:0] next_state; //next state

Always_ff @(posedge clk or posedge rst) //State Register
begin
if (rst) state <= A; //Initial state on reset
else state <= next_state; //Move to the next state on +clk edge
end
```

48

SEQUENTIAL CIRCUIT DESIGN



Creating FSMs in SystemVerilog
2. Next State Function: Combinational function determines the next state based on the current state and main inputs are the inputs of this block. Next state is the output of this function.

We can use assign statements (Combinational), but common practice is use an always@(*) or always@(state,inputs – sensitivity list) blocks to define the combinational function since it allows us to use more complex expressions such as if, and case statements.
- Describe what happens in each state
- Case statement is natural for this

```
//Next state as function of X and state
always_comb @(X or state)
begin
case (state)   // X=1 : X=0
A: next_state = X ? B : A;
B: next_state = X ? C : A;
C: next_state = X ? C : D;
D: next_state = X ? B : A;
default: next_state = A; //if not all cases are covered we need this
endcase
end
```

49

SEQUENTIAL CIRCUIT DESIGN

Creating FSMs in Verilog
always_comb @(*) or always_comb @(X or state) block

- Used for combinational logic functions

- It is always active – like assign statements

- Assignment (=) used to assign function value

- Output can be assigned more than once
  e.g. multiple if statements

  The last one counts

- All outputs must be assigned at least once

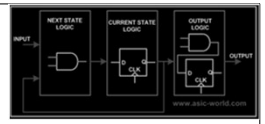
  No matter how ifs and cases are executed

  Otherwise function is undefined

  Use default assignments to help you

50

SEQUENTIAL CIRCUIT DESIGN



Creating FSMs in Verilog
3. Output Function: Combinational function determines the outputs based on the current state and main inputs which are the inputs of this block. Main outputs are the output of this function.

We can use assign statements (Combinational), but common practice is use an always@(*) or always@(state,inputs – sensitivity list) blocks to define the combinational function since it allows us to use more complex expressions such as if, and case statements
- Describe the output of each state

```
always_comb @(X or state)
begin
        case (state)
                A: Z = 1'b0;
                B: Z = 1'b0;
                C: Z = 1'b0;
                D: Z = X ? 1'b1 : 1'b0;
        default: Z = 0; //if not all cases are covered we need this
        endcase
end
endmodule
```

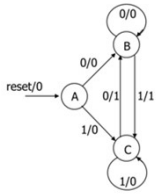
51

SEQUENTIAL CIRCUIT DESIGN

Edge Detector Example

```
localparam A=0, B=1, C=2;
reg [1:0] state,    // Current state
          nxtState; // Next state

always @(posedge clk) begin
  if (reset) begin
    state <= A;   // Initial state
  end else begin
    state <= nxtState;
  end
end
```




```
always @(*) begin
  nxtState = state;
  out = 0;
  case (state)
    A : if (in) nxtState = C;
        else    nxtState = B;
    B : if (in) begin
          out = 1;
          nxtState = C;
        end
    C : if (~in) begin
          out = 1;
          nxtState = B;
        end
    default : begin
      out = 1'bX;
      nxtState = 3'bX;
    end
  endcase
end
```

52

SEQUENTIAL CIRCUIT DESIGN

Please use “standard” FSM SystemVerilog style shown here

- Do not be beguiled into thinking this is programming C!

- always @(posedge clk) block

  - Use only for registers with simple logic
  - e.g. shifter, counter, enabled register, etc.

- Miscellaneous combinational logic

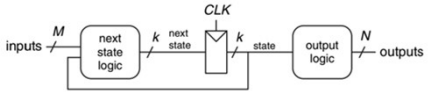
  - assign statements (always safe)

  - always @(*) block (be very careful)
    - Think of this as a complex assign statement
    - Fine to have more than one

53

FSM Örneği 1 - Problem

- Her **üç clock da bir tick** üreten bir FSM devresi tasarlayın



Girişler:

Çıkışlar:

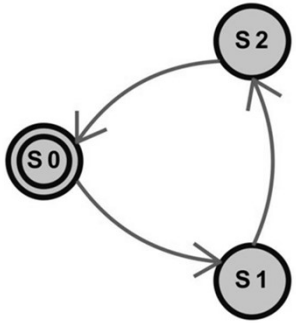
Toplam stateler:

Registerlar:

GEBZE TEKNİK ÜNİVERSİTESİ

54

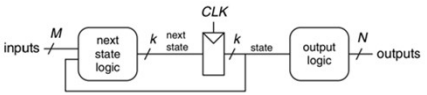
FSM Örneği 1 - State Transition Diagram



GEBZE TEKNİK ÜNİVERSİTESİ

55

FSM Örneği 1 – Current State Logic



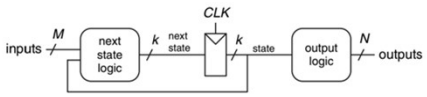

```
// state register
   always @ (posedge clk, posedge reset)
      if (reset) state <= S0;
      else       state <= nextstate;
```

- **This part defines the state register (memorizing process)**
- **Sensitive to only clk, reset**
- **In this example reset is active when ‘1’**

GEBZE TEKNİK ÜNİVERSİTESİ

56

FSM Örneği 1 - Next State Logic

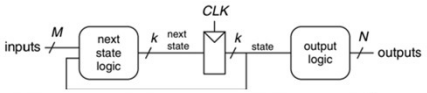



```
// next state logic
  always @ (*)
     case (state)
        S0:      nextstate = S1;
        S1:      nextstate = S2;
        S2:      nextstate = S0;
        default: nextstate = S0;
     endcase
```

- Based on the value of state we determine the value of `nextstate`
- An `always .. case` statement is used for simplicity.

FSM Örneği 1 - Son Devre

- Her üç clock da bir tick üreten bir FSM devresi tasarlayın

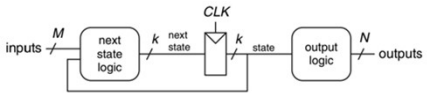



```
module divideby3FSM (input clk, input reset, output q);
   reg  [1:0] state, nextstate;

   parameter S0 = 2'b00;
   parameter S1 = 2'b01;
   parameter S2 = 2'b10;

   always @ (posedge clk, posedge reset) // state register
      if (reset) state <= S0;
      else       state <= nextstate;
   always @ (*)                          // next state logic
      case (state)
         S0:      nextstate = S1;
         S1:      nextstate = S2;
         S2:      nextstate = S0;
         default: nextstate = S0;
      endcase
   assign q = (state == S0);             // output logic
endmodule
```

FSM Örneği 1 - Output Logic




```
// output logic
   assign q = (state == S0);
```

- In this example, output depends only on state
  - Moore type FSM
- We used a simple combinational assign

FSM State Encoding Styles

- **Binary** encoding:
  - i.e., for four states, 00, 01, 10, 11
  - requires smallest number of ffs
  - it might require more combinational logic
- **Gray** encoding - same as binary
- **One-hot** encoding
  - Only one bit is high in each state
  - i.e., for 4 states, 0001, 0010, 0100, 1000
  - requires largest number of ffs
  - combinational logic is usually smaller
- **Johnson** encoding
  - Rotate bits - inverting the one that is shifted in
  - i.e., 000, 001, 011, 111, 110, 100
  - in-between the other two

FSMs in SystemVerilog

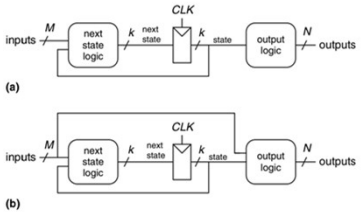
```
module fsm_tick3(
  input  logic clk, reset,
  output logic tick);

typedef enum {S0, S1, S2} statetype;
statetype state, nextstate;

// state registers
always_ff @(posedge clk)
  if (reset) state <= S0;
  else       state <= nextstate;

// next state logic
always_comb
  case (state)
    S0:      nextstate = S1;
    S1:      nextstate = S2;
    S2:      nextstate = S0;
    default: nextstate = S0;
  endcase

// output logic
assign tick = (state == S2);
endmodule
```

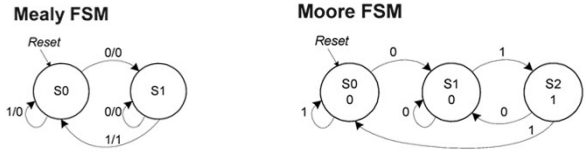


GEBZE TEKNİK ÜNİVERSİTESİ

61

Mealy vs Moore

- Genelde Mealy, Moore a göre daha az state kullanır.
- Mealy çıkışı, girişe de bağlıdır.
  - Asenkron çıkış
  - Girişlerdeki olası glitchlere hassas



GEBZE TEKNİK ÜNİVERSİTESİ

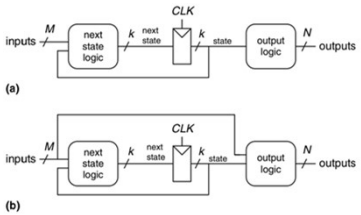
FSMs in SystemVerilog

```
module fsm_tick3_en(
  input  logic clk, reset, A,
  output logic tick);

...

// next state logic
always_comb
    case (state)
        S0:
        begin
           if (A) nextstate = S1;
           else nextstate = S0;
        end
        S1:
        begin
           if (A) nextstate = S2;
           else nextstate = S0;
        end
        S2:      nextstate = S0;
        default: nextstate = S0;
    endcase

...
```



GEBZE TEKNİK ÜNİVERSİTESİ

62

END OF LECTURE

Thanks

GEBZE TEKNİK ÜNİVERSİTESİ

64